Aegivex AI - System Architecture Document

Version: 1.0
Document Type: System Architecture Document
Project: OKX.AI Genesis Hackathon
Prepared By: Aegivex AI Team

1. Overview

This document describes the overall system architecture of Aegivex AI. The architecture is designed using a modular, scalable, and service-oriented approach to ensure maintainability, security, and future expansion. The system consists of four primary layers: Frontend Layer, Backend Layer, AI Layer, and Data Layer.

2. High-Level Architecture

```
User → Frontend (Next.js) → API Gateway → FastAPI Backend → AI Engine → Blockchain
Intelligence Services → Database → Response to User
```

3. System Components

Layer	Technology	Responsibilities
Frontend	Next.js, React, TypeScript, Tailwind CSS	Authentication, Dashboard, AI Chat Interface, Wallet Scanner UI, Token Scanner UI, Contract Scanner UI, Website Scanner UI, Transaction Explainer, Scan History
Backend	FastAPI, Python	Authentication, API Routing, Request Validation, User Management, AI Communication, Database Operations, Logging, Security
AI Layer	OpenAI API, LangChain,	Natural Language Processing, Risk Analysis, Recommendation Generation, Threat Detection,

Layer	Technology	Responsibilities
	LangGraph	Security Explanation, Prompt Orchestration
Database Layer	PostgreSQL	Stores Users, Wallet Scans, Token Scans, Contract Scans, Website Scans, Transaction Scans, Chat History, Audit Logs

4. External Services

The platform communicates with trusted third-party services:

- OpenAI API
- Blockchain RPC Providers
- Block Explorer APIs
- Token Information APIs
- Domain Reputation APIs
- Threat Intelligence APIs

5. Request Flow

1. Step 1: User submits input.
2. Step 2: Frontend validates input.
3. Step 3: Request sent to FastAPI.
4. Step 4: Backend verifies authentication.
5. Step 5: Backend collects blockchain data.
6. Step 6: AI Engine analyzes results.
7. Step 7: Risk score generated.
8. Step 8: Recommendation generated.
9. Step 9: Database stores results.
10. Step 10: Frontend displays response.

6. Authentication Flow

User Login → JWT Token Generated → Token Stored Securely → Token Sent with API Requests → Backend Validation → Authorized Access

7. Core Modules

- Module 1: Authentication Service
- Module 2: User Management
- Module 3: Wallet Scanner
- Module 4: Token Analyzer
- Module 5: Smart Contract Analyzer
- Module 6: Website Scanner
- Module 7: Transaction Explainer
- Module 8: AI Chat
- Module 9: Dashboard
- Module 10: History Service

8. Security Architecture

Security mechanisms include:

- HTTPS
- JWT Authentication
- Rate Limiting
- Input Validation
- API Key Protection
- Secure Environment Variables
- Database Encryption
- Request Logging

9. Scalability

The architecture supports future growth through:

- Modular Services
- REST APIs
- Stateless Backend
- Database Indexing
- Caching
- Horizontal Scaling
- Cloud Deployment

10. Deployment Architecture

```
Frontend (Vercel) → FastAPI Backend (Railway / Render) → PostgreSQL Database → OpenAI API
```

11. Folder Structure

Frontend Directory Structure	Backend Directory Structure
app/ components/ hooks/ services/ styles/	api/ routers/ services/ models/ schemas/ database/ prompts/ middleware/ utils/

12. Error Handling

The backend should:

- Validate requests
- Return proper HTTP status codes
- Log exceptions
- Retry AI requests when appropriate
- Display user-friendly error messages

13. Monitoring

Recommended monitoring:

- API Logs
- Error Logs
- AI Usage Metrics
- Performance Metrics
- Database Health
- User Activity

14. Future Expansion

- Multi-Agent AI System
- Browser Extension
- Mobile Application

- Enterprise Dashboard
- Multi-chain Analysis
- Real-time Wallet Monitoring
- Notification Service

15. Architecture Principles

- Modular
- Scalable
- Secure
- Maintainable
- API-First
- AI-Driven
- Cloud-Native

16. Expected Outcome

The final architecture provides a secure, scalable, and AI-powered Web3 Security Copilot capable of supporting thousands of users while maintaining high performance and a clean development structure.